Step 4 — A Timeout and a Partial-Result Fallback — Hints

> Back to the step · Hint 1 · Hint 2 · Solution

- Hint 1 — The idea, and the to_thread trap
- Hint 2 — The plan, and proving it

Hint 1 — The idea, and the to_thread trap

A tool that can hang forever is one nobody will build on. Make `deep_research` `async`, put a timeout around the pipeline, and on timeout return the best draft so far (a marked PARTIAL).

The trap: `run_pipeline` is *synchronous* (blocking LLM calls in a loop). `await asyncio.wait_for(run_pipeline(...), timeout)` would run `run_pipeline` to completion *before* `wait_for` got control — blocking the event loop, never timing out. You must run it on a thread first: `asyncio.wait_for(asyncio.to_thread(run_pipeline, ...), timeout=...)`.

Hint 2 — The plan, and proving it

```
@mcp.tool()
async def deep_research(topic):
    progress = Progress()
    try:
        result = await asyncio.wait_for(
            asyncio.to_thread(run_pipeline, MODEL, topic, cap, progress),
            timeout=CONFIG.timeout_seconds)
        return format_result(result)
    except asyncio.TimeoutError:
        return format_partial(topic, progress)   # best draft so far
```

The `Progress` holder is created here and written by the thread, so on timeout it holds whatever was done. **Known limitation:** `to_thread` can't cancel the worker thread, so the pipeline keeps running in the background after the timeout — note it; a real deployment would add a cancel flag the loop checks each round. Prove it with a tiny `DEEP_RESEARCH_TIMEOUT_SECONDS` (passed through

`StdioServerParameters(env=...)`) so even a fast run hits the PARTIAL path. The whole file set is in the Solution.